

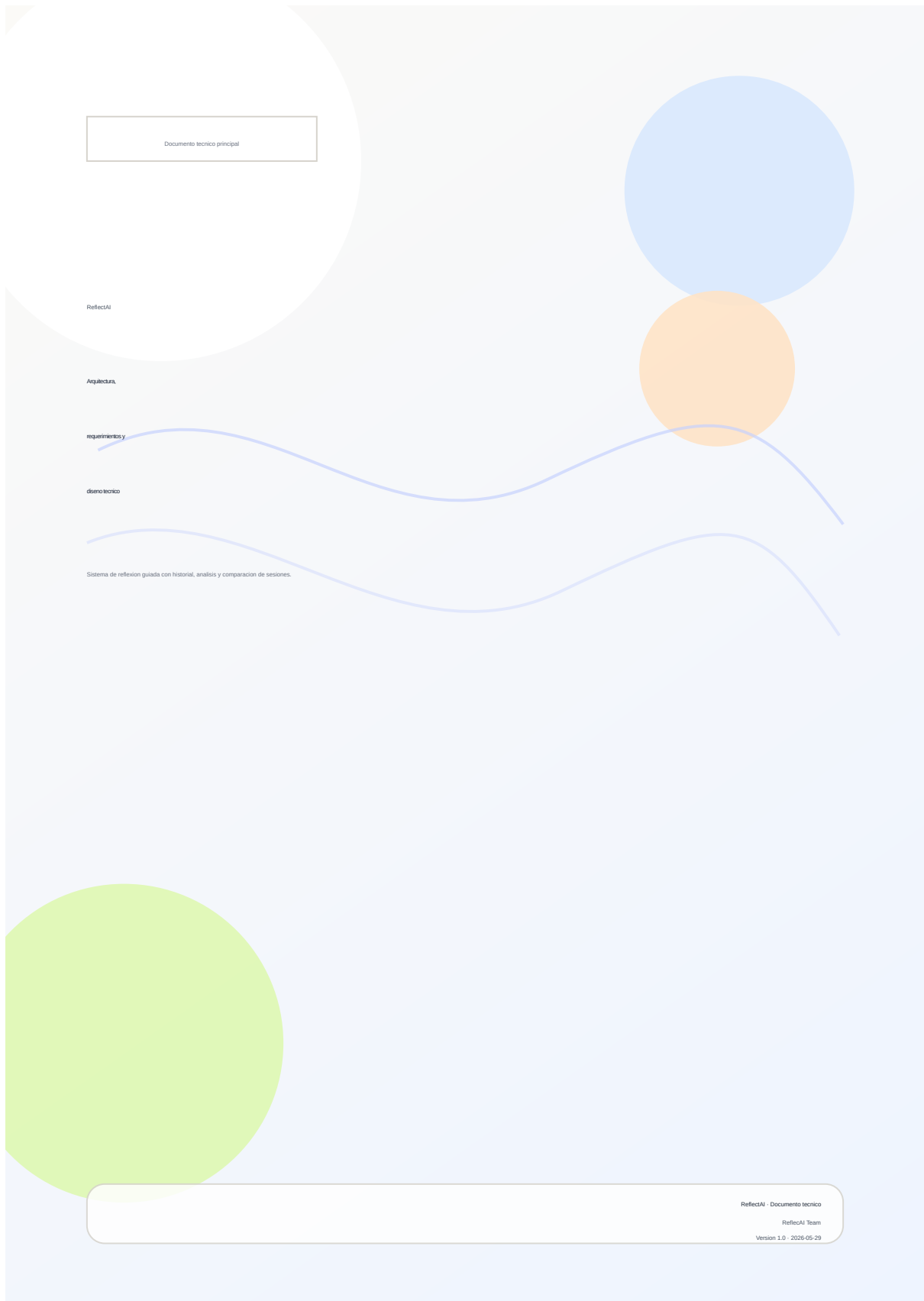


Table of Contents

1. Alcance y metodo	3
2. Resumen ejecutivo	4
3. Arquitectura final	5
3.1. Capa de presentacion	5
3.2. Capa de aplicacion y API.	5
3.3. Persistencia y datos.	5
3.4. Integraciones externas.	5
3.5. Operacion y entrega	5
4. Evaluacion de RNF.	6
4.1. RNF-01 Seguridad.	6
4.2. RNF-02 Privacidad de datos	6
4.3. RNF-03 Rendimiento	6
4.4. RNF-04 Usabilidad	7
4.5. RNF-05 Disponibilidad.	7
5. Evaluacion de ADR.	8
5.1. ADR-001	8
5.2. ADR-002	8
6. Deuda tecnica.	9
7. Riesgos y limitaciones	10
8. Runbook	11
8.1. 1. Preparacion del entorno	11
8.2. 2. Arranque local	11
8.3. 3. Validacion previa a entrega	11
8.4. 4. Operacion normal	11
8.5. 5. Manejo de incidentes frecuentes	11
8.6. 6. Cierre seguro	12
9. Recomendaciones	13

Chapter 1. Alcance y metodo

Esta auditoria evalua, a partir del codigo fuente, la documentacion del repositorio y los flujos de prueba existentes, si ReflectAI cumple con los Requisitos No Funcionales (RNF) y con las decisiones arquitectonicas (ADR) definidas al inicio del proyecto.

La revision es estatico-tecnica: se inspeccionaron los documentos de contexto, la organizacion del codigo, las rutas de API, las utilidades de seguridad, los clientes de Supabase y Groq, asi como los workflows de integracion continua. No se ejecuto una prueba de carga, ni una revision de infraestructura de produccion, ni una auditoria de configuracion de Supabase/Vercel fuera del repositorio.

Fuentes principales revisadas:

- docs/requirements.adoc
- docs/architecture-decisions.adoc
- docs/design.adoc
- reflectai/src/
- reflectai/tests/
- .github/workflows/

Chapter 2. Resumen ejecutivo

El proyecto materializa la arquitectura declarada en los ADR: ReflectAI se construyo como una aplicacion web con Next.js, Supabase y Groq, organizada por features y expuesta mediante rutas de API del propio framework.

La conclusion de auditoria es que el software cumple de forma clara las decisiones arquitectonicas principales y cumple parcialmente los RNF mas exigentes. La mayor brecha no esta en el diseno funcional, sino en la falta de evidencia cuantitativa para rendimiento y disponibilidad, y en la dependencia de configuraciones externas para cerrar totalmente seguridad y privacidad.

Area	Estado	Lectura ejecutiva
RNF	Parcial	Hay controles reales de seguridad y privacidad a nivel de aplicacion, pero el rendimiento sub-500 ms y la disponibilidad 99.5% no estan demostrados con medicion ni observabilidad.
ADR	Cumple	Las dos decisiones arquitectonicas centrales quedaron reflejadas en el codigo y en la organizacion del repositorio.
Operacion	Parcial	Existe un runbook viable para operar la app, pero depende de variables de entorno y servicios administrados externos.

Chapter 3. Arquitectura final

La arquitectura final de ReflectAI no es la de un backend separado, sino la de una aplicacion Next.js con una capa de dominio por features y una capa de integracion basada en route handlers.

3.1. Capa de presentacion

La interfaz esta dividida por areas funcionales:

- `src/app/(main)` agrupa las vistas principales del usuario autenticado.
- `src/features/auth`, `src/features/reflection`, `src/features/history`, `src/features/profile`, `src/features/dashboard`, `src/features/statistics` y `src/features/help` concentran las pantallas y hooks por dominio.
- `src/shared/ui` contiene componentes reutilizables de interfaz.

Esta estructura favorece el mantenimiento por capacidad de negocio y evita un monolito de UI dificil de evolucionar.

3.2. Capa de aplicacion y API

La logica servidor se expone mediante `src/app/api/*`. Ahí viven los flujos de autenticacion, perfil, sesiones de reflexion e IA.

La capa de aplicacion usa:

- validacion con zod
- respuestas tipadas y normalizadas
- control de origen confiable
- rate limiting en memoria para rutas sensibles
- verificacion de usuario autenticado antes de mutaciones o lectura de datos privados

3.3. Persistencia y datos

Supabase cumple dos funciones:

- autenticacion y sesion
- persistencia en PostgreSQL y Storage

El modelo de datos visible en el codigo usa una mezcla relacional-documental. Las sesiones de reflexion se guardan en `reflection_sessions` con un payload flexible y un resumen `ai_analysis`, mientras que el perfil y la identidad permanecen asociados al usuario autenticado.

3.4. Integraciones externas

El motor de IA se resuelve con Groq a traves de `src/lib/ai/groqClient.ts`. El codigo contempla generacion de preguntas dinamicas, analisis de sesiones y una cita diaria, con degradacion a contenido estatico cuando el servicio falla.

3.5. Operacion y entrega

El repositorio contiene workflows de GitHub Actions para ejecutar pruebas, cobertura y analisis adicional. Eso da trazabilidad de calidad, aunque la configuracion concreta de Vercel no se encuentra versionada en el repo.

Chapter 4. Evaluacion de RNF

RNF	Estado	Evidencia	Dictamen
RNF-01 Seguridad	Parcial	Supabase gestiona autenticacion y hashing de contraseñas; hay validacion de origen, rate limiting y control de acceso en rutas sensibles.	El cumplimiento es razonable en la capa de aplicacion, pero no se puede cerrar al 100% sin revisar la configuracion de Supabase, la duracion real de sesiones y la politica de cookies/headers en produccion.
RNF-02 Privacidad de datos	Parcial	Las consultas a sesiones y perfiles se filtran por user_id; la eliminacion de cuenta usa cliente admin y cierre de sesion.	La aplicacion protege los accesos en su capa server, pero no hay evidencia en el repo de politicas RLS o de hardening de buckets y tablas para asegurar el aislamiento de objeto de extremo a extremo.
RNF-03 Rendimiento	Parcial	El flujo de preguntas tiene fallback local cuando la IA no responde y la UI esta particionada por features.	No existen mediciones, budgets ni pruebas de latencia que permitan afirmar que el cambio entre preguntas ocurre siempre en menos de 500 ms.
RNF-04 Usabilidad	Cumple	La UI se organiza con componentes reutilizables, Tailwind y una experiencia mobile-first en la documentacion y el codigo.	La implementacion es consistente con el RNF: la interfaz esta pensada para consumo rapido, responsivo y con baja friccion cognitiva.
RNF-05 Disponibilidad	Parcial	Uso de servicios administrados, manejo de errores y workflows de CI.	Falta observabilidad, monitoreo, SLO/SLA y evidencia de redundancia; por eso la disponibilidad objetivo no puede darse por garantizada solo con el repo.

4.1. RNF-01 Seguridad

El proyecto toma una postura correcta al delegar autenticacion y hashing a Supabase, en lugar de implementar criptografia propia. Ademas, la capa de API agrega rate limiting, validacion de origen y control de acceso por usuario autenticado.

La limitacion principal es de auditoria, no de intencion: desde el repositorio no se puede probar que las cookies, expiraciones de JWT y reglas de despliegue en produccion esten cerradas exactamente como el RNF lo pide.

4.2. RNF-02 Privacidad de datos

La aplicacion evita accesos globales y trabaja con consultas acotadas por user_id. Eso es positivo y coherente con el tipo de dato que maneja ReflectAI, porque el contenido de las sesiones es sensible.

Lo que queda abierto es la capa de base de datos administrada. Sin revisar politicas de Supabase, no es posible afirmar que ningun cliente directo pueda ver datos ajenos si la configuracion externa no acompaña al codigo.

4.3. RNF-03 Rendimiento

La arquitectura favorece una experiencia rapida porque separa UI, dominio y acceso a

datos, y porque el flujo de preguntas cae a contenido estatico cuando la IA no esta disponible.

Sin embargo, el umbral de 500 ms requiere benchmark real. Con llamadas a Groq y a Supabase, ese objetivo depende del entorno de red y de la respuesta del proveedor, asi que hoy debe considerarse una meta de diseno, no una garantia demostrada.

4.4. RNF-04 Usabilidad

Este es el RNF mejor resuelto por la implementacion visible. La distribucion por features, los componentes reutilizables, el enfoque mobile-first y la existencia de pantallas de ayuda apuntan a una experiencia simple y consistente.

4.5. RNF-05 Disponibilidad

El sistema esta preparado para seguir operando en condiciones degradadas en algunos flujos, pero no existe en el repo una capa de observabilidad o resiliencia que permita certificar el 99.5% de uptime.

En la practica, la disponibilidad esta delegada a Supabase, Groq y Vercel, lo cual es razonable para un MVP, pero introduce dependencia de terceros.

Chapter 5. Evaluacion de ADR

ADR	Estado	Evidencia	Dictamen
ADR-001 Next.js + BaaS en lugar de backend tradicional	Cumple	reflectai/package.json, reflectai/src/app/api/ reflectai/src/lib/supabase/ y reflectai/src/lib/auth/*.	La arquitectura implementada coincide con la decision: Next.js concentra UI y backend ligero, y Supabase absorbe autenticacion y datos.
ADR-002 Almacenamiento hibrido relacional-documental	Cumple	reflection_sessions guarda payload y ai_analysis; el codigo normaliza cargas flexibles y persiste sesiones dinamicas.	La decision se materializo de forma consistente: entidades de identidad y perfil siguen el modelo relacional, mientras la sesion usa estructura flexible.

5.1. ADR-001

La evidencia del repositorio muestra una adopcion real de Next.js como marco full-stack y de Supabase como BaaS. No hay un backend tradicional separado ni una capa de servicios pesada; en su lugar hay route handlers, servicios livianos y clientes centralizados.

5.2. ADR-002

La persistencia de sesiones con payload flexible y analisis de IA confirma el modelo relacional-documental. Esto encaja especialmente bien con los flujos de reflexion, donde la cantidad y la forma de las respuestas pueden cambiar con el tiempo.

Chapter 6. Deuda tecnica

- Falta un plan de observabilidad: no se ve trazabilidad de latencia, error rate, disponibilidad ni alertas.
- El rate limiting vive en memoria, por lo que no escala bien a multiples instancias o despliegues distribuidos.
- La seguridad completa depende de configuracion externa de Supabase, bucket policias y variables de entorno; esa configuracion no esta documentada como codigo.
- Groq es un punto de dependencia externo para la parte dinamica del valor del producto.
- El modelo de respuestas flexibles es correcto, pero complica consultas analiticas avanzadas si el volumen de datos crece.
- No se observan pruebas de carga ni baselines de rendimiento para el objetivo de 500 ms.

Chapter 7. Riesgos y limitaciones

- Riesgo de vendor lock-in en autenticacion, storage y despliegue.
- Riesgo de experiencia degradada cuando la IA o la red externa respondan lento.
- Riesgo de privacidad si la configuracion de RLS en Supabase no acompaña las restricciones que el código asume.
- Riesgo operacional por depender de secretos de entorno y de servicios de terceros sin un runbook de incidentes mas profundo.
- Limitacion del audit: esta revision no incluye pruebas de penetracion ni validacion sobre un ambiente productivo real.

Chapter 8. Runbook

8.1. 1. Preparacion del entorno

Verificar que existan las variables de entorno requeridas para Next.js y Supabase:

- NEXT_PUBLIC_SUPABASE_URL
- NEXT_PUBLIC_SUPABASE_ANON_KEY
- SUPABASE_SERVICE_ROLE_KEY
- NEXT_PUBLIC_SITE_URL
- GROQ_API_KEY
- GROQ_MODEL opcional

Si el flujo de correos o recovery depende de otros secretos en el entorno de despliegue, deben agregarse antes de publicar.

8.2. 2. Arranque local

En reflectai/ ejecutar:

- npm install
- npm run dev

La aplicacion queda disponible en desarrollo mediante Next.js. Para una verificacion mas cercana a produccion:

- npm run build
- npm run start

8.3. 3. Validacion previa a entrega

Antes de liberar una version conviene ejecutar:

- npm run lint
- npm run typecheck
- npm run test:unit
- npm run test:integration
- npm run test:regression

Si se va a revisar cobertura o calidad en pipeline, los workflows de GitHub Actions ya contemplan esos pasos.

8.4. 4. Operacion normal

El flujo operacional esperado es:

- El usuario se registra o inicia sesion.
- Supabase establece la autenticacion.
- El usuario inicia una sesion de reflexion.
- Las respuestas se guardan en reflection_sessions.
- Groq genera preguntas o analisis cuando el flujo lo requiere.
- El historial y las estadisticas se consultan desde las vistas protegidas.

8.5. 5. Manejo de incidentes frecuentes

- Si falla el login, revisar primero variables de entorno, credenciales y estado de Supabase.
- Si falla la generacion de preguntas, validar GROQ_API_KEY y la respuesta del proveedor; el sistema debe caer a preguntas estaticas.
- Si falla la subida de avatar, verificar que exista el bucket profile-avatars y sus permisos.
- Si la sesion parece perdida, revisar cookies del navegador, origin trusted y expiracion de sesion.
- Si una ruta retorna 401 de forma inesperada, confirmar que el usuario siga autenticado y que la sesion no haya caducado.

8.6. 6. Cierre seguro

Para cerrar una sesion o preparar mantenimiento:

- completar o guardar el estado de la sesion activa
- cerrar la sesion de Supabase
- confirmar que no queden procesos de desarrollo ejecutandose
- registrar cualquier error recurrente para analisis posterior

Chapter 9. Recomendaciones

- Formalizar RLS, policies y permisos de Storage como parte del código o de IaC revisable.
- Medir tiempos de respuesta reales para el flujo dinámico de preguntas y fijar un budget verificable.
- Cambiar el rate limiting en memoria por una estrategia compartida si se piensa escalar horizontalmente.
- Agregar monitoreo y alertas basados en error rate, latencia y disponibilidad.
- Documentar el inventario de variables de entorno y secretos por ambiente.